Q1a) How does Amdahl's Law limit the potential speedup of a parallelized program?

Ans 1a) 1. Introduction:

- **Amdahl's Law** is a principle in parallel computing that **predicts the maximum speedup** achievable by **parallelizing a program**.
- It was formulated by **Gene Amdahl (1967)**.
- The law considers that a program has:
  - A **serial portion** (1−P1 - P1−P) that **cannot be parallelized**
  - A **parallel portion** (PPP) that **can be executed simultaneously** on multiple processors.

---

2. Formula:

Speedup(S)=1(1−P)+PN\text{Speedup} (S) = \frac{1}{(1-P) + \frac{P}{N}}Speedup(S)=(1−P)+NP1

Where:

- PPP = fraction of the program that can be parallelized
- 1−P1-P1−P = fraction that is inherently serial
- NNN = number of processors

---

3. Explanation:

1. **Serial Limitation:**
   - The **serial portion** of a program cannot benefit from parallelism.
   - Even with **infinite processors**, this serial part limits the speedup.
2. **Parallel Portion:**
   - The **parallel portion** is divided among NNN processors.
   - Speedup increases as NNN increases, but the **serial part dominates at high N**.
3. **Maximum Speedup:**
   - If N→∞N \to \inftyN→∞:

Smax=11−PS_\text{max} = \frac{1}{1-P}Smax=1−P1

- This shows that the **serial fraction** of the program is the **ultimate limiting factor**.

---

4. Example:

- Suppose **80% of a program** is parallelizable (P=0.8P = 0.8P=0.8) and 20% is serial (1−P=0.21-P = 0.21−P=0.2).

- Maximum speedup, even with infinite processors:

$$S_{max} = \frac{1}{0.2} = 5$$

- This means the program cannot run faster than **5 times the original speed**, no matter how many processors are added.

---

5. Conclusion:

- **Amdahl's Law** shows that:
  - Speedup is limited by the **non-parallelizable portion** of a program.
  - Adding more processors yields **diminishing returns**.
- To achieve higher performance, programs must **reduce serial bottlenecks**.

Q1b) What are the trade-off's between CISC (Complex Instruction Set Computing) and RISC (Reduced Instruction Set Computing) architectures?

Ans 1b) 1. Introduction:

- **CISC** and **RISC** are two types of **CPU design philosophies**.
- They differ in **instruction set complexity, execution speed, and hardware design**.
- Understanding the trade-offs helps in **choosing architecture based on application requirements**.

---

2. Comparison Table: CISC vs RISC

Feature	CISC (Complex Instruction Set Computing)	RISC (Reduced Instruction Set Computing)
Instruction Set	Large and complex instructions	Small and simple instructions
Instruction Length	Variable-length instructions	Fixed-length instructions
Execution Time	Instructions may take multiple cycles	Most instructions execute in **1 clock cycle**
Microcode	Uses **microcode** to implement complex instructions	Hardwired control, minimal microcode
Memory Usage	Fewer instructions needed → smaller programs	More instructions → potentially larger programs
Hardware Complexity	Complex hardware to decode and execute instructions	Simpler hardware, easier to pipeline
Examples	Intel x86, VAX	ARM, MIPS, SPARC

Feature	CISC (Complex Instruction Set Computing)	RISC (Reduced Instruction Set Computing)
Pipeline Efficiency	Difficult to pipeline due to variable instruction length	Highly pipelined due to fixed-length instructions
Compiler Role	Less dependent on compiler	Compiler does more work to optimize instruction usage
Speed	High performance per instruction, but slower clock	Higher clock frequency possible, more instructions may be needed

---

3. Trade-offs / Design Considerations:

1. **Instruction Complexity vs Execution Time:**
   - CISC executes **more work per instruction**, fewer instructions per program.
   - RISC executes **simpler instructions** faster, may need more instructions.
2. **Hardware vs Compiler Dependency:**
   - CISC relies on **hardware** to implement complex instructions.
   - RISC relies on **compiler optimization** for efficient use of simple instructions.
3. **Pipelining and Parallelism:**
   - RISC is **easier to pipeline**, supporting high clock rates and parallel execution.
   - CISC's variable-length instructions complicate **pipelining** and parallelism.
4. **Memory Usage:**
   - CISC programs are **smaller** due to compact complex instructions.
   - RISC programs may be **larger**, but can execute faster due to simpler instructions.
5. **Application Suitability:**
   - **CISC**: Legacy applications, desktops, systems where **memory is a constraint**.
   - **RISC**: Embedded systems, smartphones, high-performance processors where **speed and pipelining** are important.

---

4. Conclusion:

- **CISC vs RISC trade-offs** involve **instruction complexity, execution speed, hardware complexity, and compiler reliance**.
- Modern CPUs often use **hybrid approaches**, combining **RISC pipelines with CISC instructions**, e.g., Intel x86 processors internally use **RISC-like micro-operations**.

Q2a) How does the memory hierarchy take advantage of the different speeds of various memory types to optimize performance?

Ans 2a) 1. Introduction:

- **Memory hierarchy** organizes memory types in a **pyramid** based on **speed, cost, and size**.
- Faster memory is **more expensive** and smaller, while slower memory is **cheaper** and larger.
- Goal: **maximize system performance** by keeping frequently used data in faster memory.

---

2. Memory Hierarchy Structure:

Level	Memory Type	Speed	Size	Cost
1	CPU Registers	Fastest	Few bytes	Very high
2	L1 Cache	Very fast	KBs	High
3	L2 / L3 Cache	Fast	MBs	Medium
4	Main Memory (RAM)	Moderate	GBs	Low
5	Secondary Storage	Slow	TBs	Very Low

---

3. Principle of Locality:

Memory hierarchy works based on **locality of reference**:

1. **Temporal Locality:**
   - Recently accessed data is likely to be used again.
   - Cached in **L1/L2** to reduce access time.
2. **Spatial Locality:**
   - Data near recently accessed data is likely to be used.
   - Prefetching data blocks into cache improves performance.

---

4. Optimization Mechanism:

- When CPU requests data:
  1. **Check faster memory first** (registers → L1 → L2 → RAM → disk).
  2. If data is found (**cache hit**) → fast access.
  3. If not found (**cache miss**) → retrieve from slower memory, store a copy in cache for future use.
- This **minimizes average access time**:

$$T_{avg} = h_1 t_1 + (1-h_1) h_2 t_2 + ... + (1-h_1)(1-h_2)... t_n$$

Where:

- hih_ihi = hit rate at level iii
- tit_iti = access time at level iii

---

5. Example:

- CPU needs data stored in **RAM (100 ns)** but also has **L1 cache (1 ns)**.
- Hit rate in L1 cache = 90%
- Average access time:

Tavg=0.9·1+0.1·100≈10.9 nsT_\text{avg} = 0.9 \cdot 1 + 0.1 \cdot 100 \approx 10.9 \text{ ns}Tavg=0.9·1+0.1·100≈10.9 ns

- **Significant improvement** compared to accessing main memory every time.

---

6. Conclusion:

- **Memory hierarchy** exploits **different speeds** of memory types to optimize system performance.
- By **keeping frequently used data in faster memory** and **prefetching data based on locality**, the **average memory access time is reduced**, improving CPU efficiency.
- This principle is **essential for modern high-performance architectures**.

Q2b) How has the slowing down of Moore's Law impacted processor design and performance improvements?

Ans 2b) 1. Introduction:

- **Moore's Law** states that the **number of transistors on a chip doubles approximately every 18–24 months**, historically enabling **higher clock speeds, more cores, and better performance**.
- Recently, **Moore's Law has slowed** due to **physical, thermal, and economic limitations**.
- This has significant implications on **processor design and performance improvements**.

---

2. Impact on Processor Design:

1. **Limits to Clock Speed Increase:**
    - Earlier, performance was improved by **higher clock rates**.

    - Slower transistor scaling leads to **heat dissipation and power constraints**, limiting frequency scaling.
2. **Shift to Multicore Architectures:**
    - Performance improvements now rely on **parallelism** rather than faster clocks.
    - Processors include **multiple cores**, requiring **software to exploit parallel execution**.
3. **Emphasis on Architectural Optimizations:**
    - Designers focus on **instruction-level parallelism (ILP)**: pipelining, superscalar execution, out-of-order execution.
    - More use of **specialized accelerators** (GPU, AI cores, vector units) to enhance performance for specific workloads.
4. **Energy-Efficient Designs:**
    - With power scaling limits, emphasis is on **energy-efficient processors** rather than just raw speed.
    - Techniques like **dynamic voltage and frequency scaling (DVFS)** are used.
5. **Increased Importance of Memory Hierarchy:**
    - Since CPU speed cannot grow indefinitely, **faster caches and optimized memory hierarchies** become critical to reduce bottlenecks.

---

3. Impact on Performance Improvements:

- **Performance gains are no longer automatic** from transistor scaling.
- **Software must be parallelized** to exploit multicore processors.
- **Heterogeneous computing** (mix of CPUs, GPUs, TPUs) is used to achieve higher performance.
- **Custom accelerators** for AI, cryptography, or graphics help maintain performance improvements.

---

4. Summary Table:

Aspect	Before Moore's Law Slowed	After Slowing
Clock Speed	Main driver of performance	Limited due to power/heat
Transistor Scaling	Rapid (Moore’s Law valid)	Slower, economic/physical limits
Architecture	Single-core, simple	Multicore, complex, heterogeneous
Performance Improvement	Automatic via transistor count	Requires parallelism & accelerators
Power & Energy	Less critical	Major design constraint

---

5. Conclusion:

- Slowing Moore's Law has **shifted focus from frequency scaling to parallelism, architecture, and specialization**.

- Modern processor design relies on **multicore, energy-efficient, and heterogeneous architectures**.
- Continued performance improvement now depends on **software optimization, accelerators, and system-level innovations** rather than transistor count alone.

Q3a) What are the principles of temporal and spatial locality, and how do they influence memory hierarchy design?

Ans 3a) 1. Introduction:

- **Locality of reference** is a key principle in **memory hierarchy design**.
- Programs tend to access a **small portion of memory repeatedly** (temporal) or **memory addresses close to each other** (spatial).
- Understanding locality helps **optimize cache and memory performance**.

---

2. Temporal Locality:

- **Definition:** If a memory location is **accessed once**, it is likely to be **accessed again in the near future**.
- **Implication:**
  - Frequently used data should be stored in **fast, small memory** (e.g., L1 cache, registers).
  - **Caching** leverages temporal locality by **keeping recently used data** close to the CPU.

Example:

```
for (int i=0; i<1000; i++) {
    sum += array[5]; // Accessing array[5] repeatedly
}
```

- `array[5]` exhibits **temporal locality**.

---

3. Spatial Locality:

- **Definition:** If a memory location is **accessed**, nearby memory locations are likely to be accessed **soon after**.
- **Implication:**
  - Memory is fetched in **blocks or cache lines**, not single words.
  - **Prefetching and sequential access optimizations** exploit spatial locality.

Example:

```
for (int i=0; i<1000; i++) {
    sum += array[i]; // Accessing consecutive memory addresses
}
```

- Consecutive `array[i]` accesses exhibit **spatial locality**.

---

4. Influence on Memory Hierarchy Design:

1. **Cache Design:**
   - Temporal locality → Keep recently used data in **small, fast caches**.
   - Spatial locality → Fetch **blocks of memory** into cache lines.
2. **Block Size Selection:**
   - Larger blocks exploit spatial locality but may increase **cache miss penalty**.
3. **Prefetching Mechanisms:**
   - Predict future accesses using **spatial locality** to pre-load data into cache.
4. **Replacement Policies:**
   - **LRU (Least Recently Used)** uses temporal locality to evict **least recently accessed blocks**.
5. **Memory Hierarchy Levels:**
   - Registers → L1 → L2 → Main Memory → Disk
   - Exploit **temporal locality** in smaller, faster levels and **spatial locality** by transferring contiguous data blocks.

---

5. Summary Table:

Principle	Definition	Example	Influence on Design
Temporal Locality	Recently accessed data will be used again	Repeated access to array[5]	Keep recently used data in cache
Spatial Locality	Nearby memory locations likely accessed next	Iterating array sequentially	Fetch blocks of contiguous memory

---

6. Conclusion:

- **Temporal and spatial locality** are **fundamental to memory hierarchy efficiency**.
- They guide **cache size, block size, replacement policy, and prefetching**.
- Proper exploitation of locality **reduces average memory access time**, improving CPU performance.

Q3b) Explain the process of page table lookup and how multi-level page tables help optimize memory management.

Ans 3b) 1. Introduction:

- Modern operating systems use **virtual memory**, where **logical addresses** are mapped to **physical addresses**.
- The **page table** stores these mappings: **Page Number → Frame Number**.
- **Page table lookup** is essential for translating addresses during program execution.

---

2. Page Table Lookup Process:

1. **Logical Address Breakdown:**
    - Logical address is divided into:
        - **Page Number (p)** → identifies the page in virtual memory
        - **Page Offset (d)** → offset within the page
2. **Lookup Steps:**

1. **CPU generates logical address**.
2. **Page number (p)** is used to index the **page table**.
3. **Page table entry (PTE)** provides the **frame number (f)**.
4. **Physical address** is formed by concatenating:

Physical Address=Frame Number (f) || Page Offset (d)\text{Physical Address} = \text{Frame Number (f)} \ || \ \text{Page Offset (d)}Physical Address=Frame Number (f) || Page Offset (d)

3. **Example:**

- Page size = 4 KB → 12-bit offset, 20-bit page number
- Logical address: 0x12345 → Page Number = 0x12, Offset = 0x345
- Page table lookup: Page 0x12 → Frame 0x7A
- Physical address = 0x7A345

4. **Translation Lookaside Buffer (TLB):**
    - **Caches recent page table entries** to reduce lookup time.

---

3. Multi-Level Page Tables:

- **Problem with single-level page tables:**
    - For large virtual address spaces, single-level tables require **huge memory** (even for unused pages).
- **Solution: Multi-level page tables**

1. **Divide the page number** into multiple parts: e.g., **2-level table** → Page Number = [p1 | p2]
2. **First-level table** → points to **second-level tables**
3. **Second-level table** → points to the **frame in physical memory**

Advantages:

- Reduces memory usage → allocate second-level tables **only when needed**.
- Efficient for **sparse address spaces**.
- Works well with **demand paging**.

---

4. Example of 2-Level Page Table:

- Virtual address (32-bit) → 10-bit first-level, 10-bit second-level, 12-bit offset
- Lookup process:
    1. First 10 bits → index **first-level table** → get pointer to **second-level table**
    2. Next 10 bits → index **second-level table** → get **frame number**
    3. Last 12 bits → **offset** → physical address

---

5. Summary Table:

Feature	Single-Level Page Table	Multi-Level Page Table
Memory Usage	Large for big address spaces	Smaller, allocate second-level tables on demand
Address Translation Time	1 lookup + TLB	Multiple lookups (can use TLB)
Suitability	Small address spaces	Large/sparse address spaces

---

6. Conclusion:

- **Page table lookup** translates **logical addresses to physical addresses** using **page number and offset**.
- **Multi-level page tables** optimize memory by **reducing table size** and handling **sparse virtual address spaces** efficiently.
- Combined with **TLB**, this ensures **fast and memory-efficient address translation**.

Q4a) How does set-associative cache organization balance the trade-offs between speed and complexity? A processor has a cache with a hit rate of 90%. The time to access data from the cache is 10 ns, and the time to access data from the main memory (on a miss) is 100 ns. What is the average memory access time?

Ans 4a) 1. Introduction to Cache Organization:

- **Cache memory** reduces the **average memory access time** by storing frequently used data close to the CPU.

- Cache organization types:
    1. **Direct-mapped cache** – simple, fast, but may have many conflicts.
    2. **Fully associative cache** – flexible, fewer conflicts, but complex and slower.
    3. **Set-associative cache** – **hybrid approach** balancing speed and complexity.

---

2. Set-Associative Cache Organization:

- **Definition:** Cache is divided into **sets**, each containing **multiple lines (ways)**.
- A memory block maps to **one set**, but can occupy **any line in that set**.
- **Trade-offs:**

Feature	Effect in Set-Associative Cache
Speed	Faster than fully associative because fewer lines per set to search.
Complexity	Less complex than fully associative; requires small comparators per set.
Conflict Misses	Reduced compared to direct-mapped cache.

- Common organizations: **2-way, 4-way, 8-way set-associative**.

---

3. Average Memory Access Time (AMAT):

- **Formula:**

AMAT=(Hit Rate×Cache Access Time)+(Miss Rate×Memory Access Time)\text{AMAT} = (\text{Hit Rate} \times \text{Cache Access Time}) + (\text{Miss Rate} \times \text{Memory Access Time})AMAT=(Hit Rate×Cache Access Time)+(Miss Rate×Memory Access Time)

- **Given:**
    - Hit rate = 90% → 0.9
    - Cache access time = 10 ns
    - Memory access time = 100 ns → miss penalty = 100 ns
- **Miss rate:**

Miss Rate=1−0.9=0.1\text{Miss Rate} = 1 - 0.9 = 0.1Miss Rate=1−0.9=0.1

- **AMAT Calculation:**

AMAT=(0.9×10)+(0.1×100)\text{AMAT} = (0.9 \times 10) + (0.1 \times 100)AMAT=(0.9×10)+(0.1×100) AMAT=9+10=19 ns\text{AMAT} = 9 + 10 = 19 \text{ ns}AMAT=9+10=19 ns

---

4. Conclusion:

1. **Set-associative cache** offers a **good balance** between speed and complexity:
   - Faster than fully associative cache.
   - Fewer conflict misses than direct-mapped cache.
2. **Average memory access time** with 90% hit rate:

19 ns\boxed{19 \text{ ns}}19 ns

- Using **set-associative caches**, systems achieve **high hit rates** without excessive hardware complexity, optimizing overall performance.

Q4b) In a system, the Effective Memory Access Time (EMAT) is measured to be 105 cycles. (CO 2) The system has:
TLB Access Time: 2 cycles, Memory Access Time: 80 cycles, Page Table Access Time: 80 cycles Calculate the TLB Hit Rate.

Ans 4b) 1. Introduction:

- **Translation Lookaside Buffer (TLB)** is a cache for **page table entries**.
- **TLB hit** → page number is found in TLB → only **TLB + memory access** required.
- **TLB miss** → page table must be accessed → requires **TLB + page table + memory access**.

---

2. Formula for Effective Memory Access Time (EMAT):

EMAT=(TLB hit rate)×(TLB access + memory access)+(TLB miss rate)×(TLB access + page table access + memory access)\text{EMAT} = (\text{TLB hit rate}) \times (\text{TLB access + memory access}) + (\text{TLB miss rate}) \times (\text{TLB access + page table access + memory
access})EMAT=(TLB hit rate)×(TLB access + memory access)+(TLB miss rate)×(TLB access + page table access + memory access)

Let:

- hhh = TLB hit rate
- 1−h1-h1−h = TLB miss rate
- Memory access time = M=80M = 80M=80 cycles
- TLB access time = T=2T = 2T=2 cycles
- Page table access = P=80P = 80P=80 cycles

EMAT=h·(T+M)+(1−h)·(T+P+M)\text{EMAT} = h \cdot (T + M) + (1-h) \cdot (T + P + M)EMAT=h·(T+M)+(1−h)·(T+P+M)

3. Substituting Values:

105=h·(2+80)+(1−h)·(2+80+80)105 = h \cdot (2 + 80) + (1-h) \cdot (2 + 80 + 80)105=h·(2+80)+(1−h)·(2+80+80) 105=h·82+(1−h)·162105 = h \cdot 82 + (1-h) \cdot 162105=h·82+(1−h)·162 105=82h+162−162h105 = 82h + 162 - 162h105=82h+162−162h 105=162−80h105 = 162 - 80h105=162−80h 80h=162−10580h = 162 - 10580h=162−105 80h=5780h = 5780h=57 h=5780=0.7125≈71.25%h = \frac{57}{80} = 0.7125 \approx 71.25\%h=8057=0.7125≈71.25%

4. Conclusion:

- The **TLB hit rate** required to achieve an EMAT of 105 cycles is approximately:

71.25%\boxed{71.25\%}71.25%

- This shows that **around 71% of memory accesses are satisfied by the TLB**, significantly reducing average memory access time.

Q5a) How does Data-Level Parallelism (DLP) differ from ILP, and what types of workloads benefit from DLP?

Ans 5a) 1. Introduction:

- **Parallelism** in computer architecture is the technique of performing **multiple operations simultaneously** to improve performance.
- Two common forms of parallelism:
    1. **Instruction-Level Parallelism (ILP)**
    2. **Data-Level Parallelism (DLP)**

2. Instruction-Level Parallelism (ILP):

- **Definition:** ILP exploits **parallel execution of independent instructions** within a single program.
- **Mechanism:**
    - Pipelining
    - Superscalar execution (multiple instructions per cycle)
    - Out-of-order execution
- **Focus: Different instructions** executed concurrently.

Example:

```
a = b + c;
d = e + f;
```

- If `a` and `d` are independent, both can be executed simultaneously in a superscalar processor.

---

3. Data-Level Parallelism (DLP):

- **Definition:** DLP exploits **parallel execution of the same operation on multiple data elements**.
- **Mechanism:**
  - Vector processors (SIMD – Single Instruction Multiple Data)
  - GPUs with hundreds of cores
- **Focus: Same instruction applied to multiple data points simultaneously**.

Example:

```
for (i=0; i<1000; i++)
    C[i] = A[i] + B[i];
```

- Same addition operation applied to multiple array elements in parallel.

---

4. Differences between ILP and DLP:

Feature	ILP	DLP
Parallelism Type	Instruction-level	Data-level
Execution	Multiple independent instructions	Same instruction on multiple data
Hardware Support	Superscalar, pipelining	SIMD units, vector processors, GPUs
Best for	General-purpose programs	Scientific computing, multimedia, AI, graphics
Example	a = b + c; d = e + f;	C[i] = A[i] + B[i];

---

5. Workloads that Benefit from DLP:

1. **Scientific Computing / Simulations**
   - Large arrays, matrices, and vectors.
2. **Graphics and Image Processing**
   - Pixel operations, shading, transformations.
3. **Machine Learning / AI Workloads**
   - Vectorized operations on tensors and matrices.

4. **Multimedia Applications**
    - Audio/video encoding, filtering, and rendering.

---

6. Conclusion:

- **ILP** exploits **independent instructions** in a program, while **DLP** exploits **repetition of the same operation across multiple data elements**.
- **DLP workloads** are highly **parallelizable and compute-intensive**, and benefit from **SIMD architectures and GPUs**, leading to significant performance improvement.

Q5b) What are the key components of modern computer architecture, and how do they interact?

Ans 5b) 1. Introduction:

- Modern computer architecture organizes hardware and software to **execute programs efficiently**.
- Key components work together to **process instructions, store data, and communicate with peripherals**.

---

2. Key Components of Modern Computer Architecture:

1. **Central Processing Unit (CPU):**
    - **Control Unit (CU):** Decodes instructions and controls data flow.
    - **Arithmetic Logic Unit (ALU):** Performs arithmetic and logical operations.
    - **Registers:** Fast storage for temporary data and instruction operands.
2. **Memory Hierarchy:**
    - **Cache (L1, L2, L3):** Fast, small memory for frequently accessed data.
    - **Main Memory (RAM):** Stores currently running programs and data.
    - **Secondary Storage (SSD/HDD):** Stores programs and data persistently.
3. **Input/Output (I/O) Units:**
    - Interfaces for **peripherals** like keyboard, mouse, display, network.
4. **Buses and Interconnects:**
    - **Data bus, address bus, control bus** for communication between CPU, memory, and I/O.
    - High-speed interconnects in modern architectures (e.g., **PCIe, NVLink**).
5. **Specialized Units (Optional / Modern Enhancements):**
    - **GPU / Vector Units:** For data-parallel workloads.
    - **TPU / AI accelerators:** For machine learning tasks.
    - **DMA (Direct Memory Access):** Allows peripherals to access memory without CPU intervention.

3. Interaction Among Components:

1. **Instruction Execution:**
   - CPU fetches instructions from memory via **instruction bus**.
   - Control Unit decodes and sends signals to ALU and registers.
2. **Data Movement:**
   - ALU performs computation using data from registers or cache.
   - Results stored back in registers or memory.
3. **Memory Hierarchy Usage:**
   - Frequently accessed data in **registers or cache** → fast access.
   - Less frequently used data in **main memory** → slower access.
   - Secondary storage accessed via OS only when needed.
4. **I/O Interaction:**
   - CPU communicates with I/O devices through **controllers and buses**.
   - DMA allows large data transfers without blocking CPU.
5. **Parallelism and Pipelining:**
   - Modern CPUs use **instruction pipelining** and **multiple execution units** to overlap fetch, decode, execute stages.
   - Specialized units like GPU interact with main memory through high-speed interconnects to accelerate DLP workloads.

---

4. Summary Table:

Component	Function	Interaction
CPU (ALU, CU, Registers)	Executes instructions, controls flow	Reads/writes memory, communicates via buses
Cache / Memory	Stores instructions/data	Provides fast access to CPU
Secondary Storage	Persistent data storage	Accessed via memory hierarchy
I/O Units	Input/output operations	Communicates via buses & controllers
Buses / Interconnect	Data and control transfer	Connects CPU, memory, I/O
Specialized Units	Accelerate specific workloads	Interact with memory and CPU

---

5. Conclusion:

- Modern computer architecture consists of **CPU, memory hierarchy, I/O, buses, and specialized units**.
- Components interact through **buses and control signals**, with **memory hierarchy and caches** optimizing performance.
- **Pipelining, parallelism, and accelerators** further enhance the speed and efficiency of computation.